
ICD - INTRO. A CIENCIA DE DADOS E APRENDIZADO DE MAQUINA

PARTE 3 - Complemento: Redes Neurais Artificiais

Prof. Ramon Mayor Martins, Dr.

ramon.mayor at: ifsc.edu.br

Agenda

Conceitos Complementares

- Overfitting vs Underfitting
- Forward Pass (Propagacao para Frente)
- Backward Pass (Retropropagacao do Erro)

Hiperparametros da Rede Neural

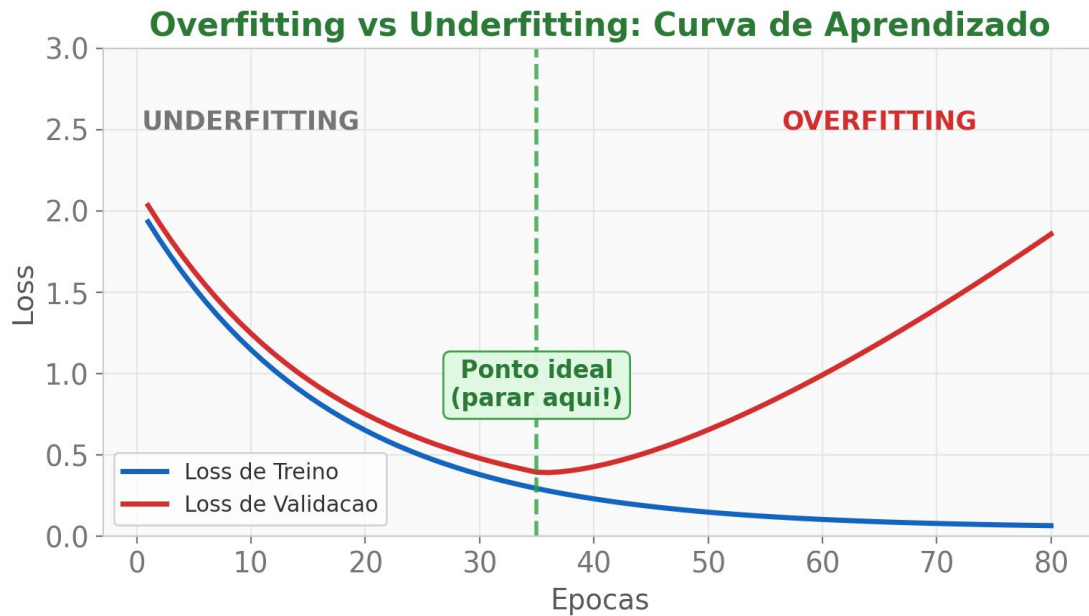
- Taxa de Aprendizado (Learning Rate)
- Batch Size e Numero de Epocas
- Otimizador e Momentum
- Funcoes de Ativacao
- Arquitetura da Rede
- Regularizacao (Dropout, L1/L2)

[Ferramentas Interativas: rmayormartins.github.io/tools-ia-projects](https://rmayormartins.github.io/tools-ia-projects)

Conceitos Complementares

Overfitting, Underfitting, Forward Pass e Backward Pass

Overfitting vs Underfitting



Underfitting:

Modelo nao aprendeu os padroes. Poucas epocas ou modelo simples demais. Erro alto no treino e no teste.

Ponto Ideal:

Loss de validacao no minimo. Modelo generaliza bem para dados novos.

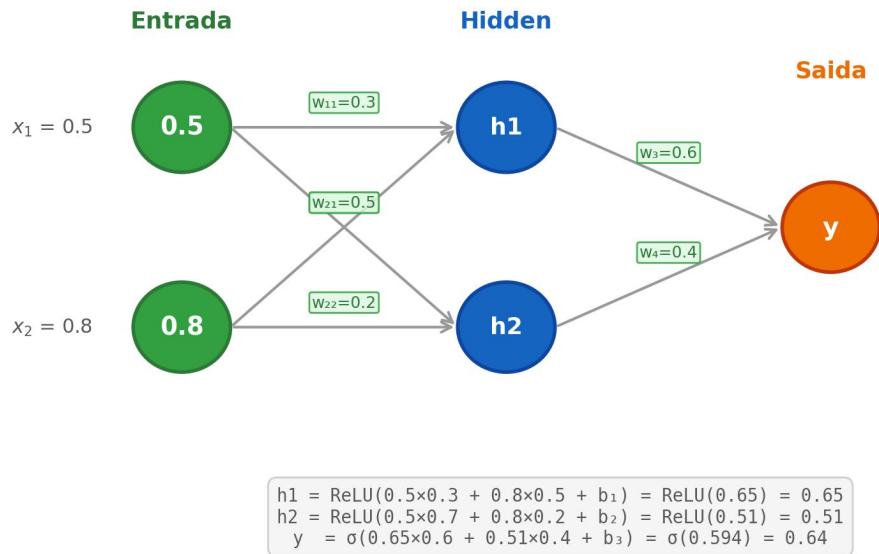
Overfitting:

Modelo decorou os dados de treino mas erra nos novos. Loss de validacao sobe enquanto a de treino desce.

Solucoes:

- Early stopping
- Dropout / Regularizacao
- Mais dados de treinamento

Forward Pass (Propagacao para Frente)



O que e?

O caminho dos dados da entrada ate a saida. A rede calcula o resultado final.

Passo 1: Entradas

$x_1 = 0.5$, $x_2 = 0.8$

Passo 2: Hidden Layer

$h = \text{ReLU}(\text{entrada} \times \text{peso} + \text{bias})$

Passo 3: Saida

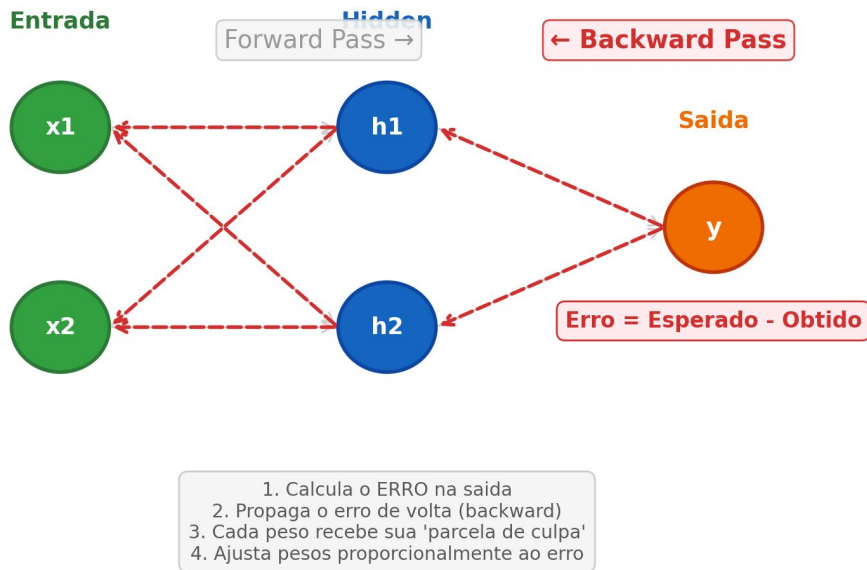
$y = \text{Sigmoid}(h \times \text{peso} + \text{bias})$

Resultado:

64% probabilidade de ser gato!

[Ferramentas: Neural Network Architecture | MLP Visualization](#)

Backward Pass (Retropropagacao / Backpropagation)



[Ferramenta: Backpropagation Simulator](#)

O que é?

O caminho inverso: o erro volta da saída até a entrada, ajustando os pesos.

Como funciona?

1. Calcula o erro na saída (loss)
2. Propaga o erro de volta pela rede
3. Cada peso recebe sua 'parcela de culpa' (gradiente)
4. Atualiza pesos na direção que reduz o erro

Analogia:

Errou a questão na prova → professor mostra onde errou → aluno corrige o raciocínio de trás pra frente.

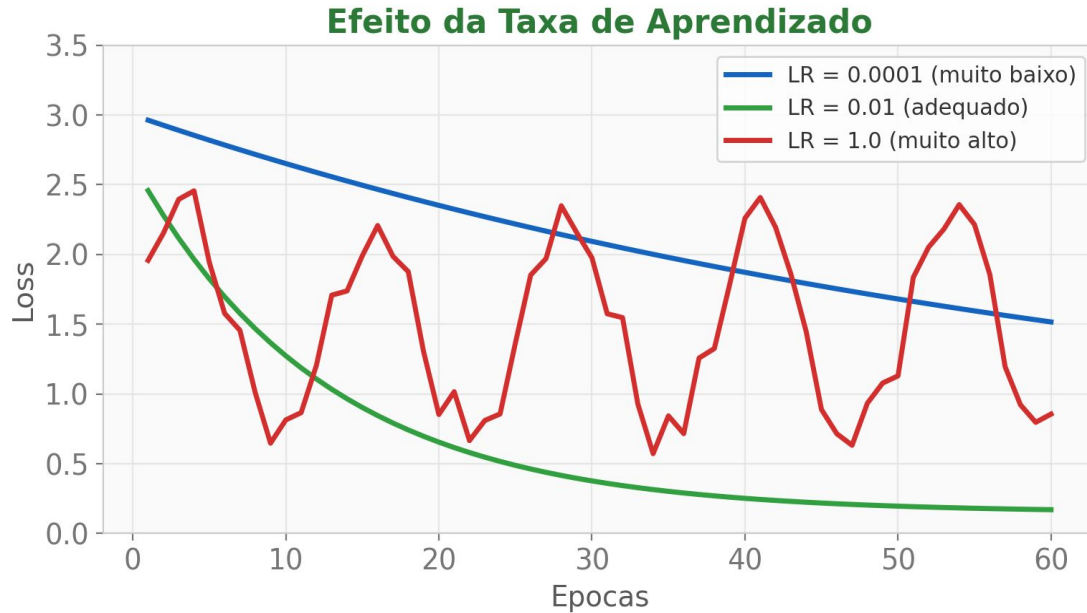
Função de Perda (Loss):

Cross-Entropy para classificação. Mede o quanto a previsão se distanciou do correto.

Principais Hiperparametros da Rede Neural

Parametros definidos ANTES do treinamento que controlam como o modelo aprende

Taxa de Aprendizizado (Learning Rate)



Muito baixo ($\eta = 0.0001$)

Treino lento, pode não convergir no tempo disponível.

Adequado ($\eta = 0.01$)

Converge bem. Encontra bom equilíbrio entre velocidade e estabilidade.

Muito alto ($\eta = 1.0$)

O modelo diverge: oscila e não aprende. Loss fica instável.

[Ferramenta: Learning Rate Simulator](#)

Batch Size e Numero de Epocas

Batch Size (Tamanho do Lote)

Quantos exemplos o modelo ve antes de atualizar os pesos.

batch_size = 1 (Estocastico / SGD)

Atualiza a cada exemplo. Ruidoso mas pode generalizar melhor.

batch_size = 32 (Mini-batch) ← Padrao

Bom equilibrio entre velocidade e estabilidade.

batch_size = N (Batch completo)

Ve tudo antes de atualizar. Estavel mas lento e exige mais memoria.

Numero de Epocas (Epochs)

Quantas vezes o modelo percorre todo o dataset de treinamento.

Poucas epocas:

Modelo nao aprendeu o suficiente (underfitting).

Epocas adequadas:

Modelo generaliza bem. Parar no ponto ideal (early stopping).

Muitas epocas:

Modelo decora o treino (overfitting).

Dica: monitorar a loss de validacao para saber quando parar.

```
model.fit(X_train, y_train, batch_size=32, epochs=50, validation_split=0.2)
```

Otimizador e Momentum

Otimizador (Optimizer)

Algoritmo que define como os pesos são atualizados para reduzir a loss.

SGD (Stochastic Gradient Descent)

Básico: segue o gradiente diretamente. Simples mas pode ser lento e oscilar.

Adam (Adaptive Moment Estimation)

Combina momentum + taxas adaptativas por peso. Mais rápido e robusto. Padrão recomendado.

RMSprop

Adapta a taxa por peso. Bom para redes recorrentes (RNN).

Momentum

Ajuda a acelerar o SGD na direção correta e a reduzir oscilações.

Analogia:

Uma bola descendo uma colina acumula velocidade. Se estava indo na direção certa, acelera. Se bateu em um obstáculo (mínimo local), a inércia ajuda a ultrapassar.

Sem momentum:

Pode oscilar em torno do mínimo.

Com momentum (0.9):

Converge mais rápido e com menos oscilação.

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Funcoes de Ativacao

Definem a saida de cada neuronio. Sem elas, a rede seria apenas uma combinacao linear (uma reta).

ReLU

Hidden Layers

$$\max(0, x)$$

Padrao da industria. Simples: se negativo, 0; se positivo, passa. Resolve vanishing gradient. Computacionalmente rapida.

Sigmoid

Saida Binaria

$$1 / (1 + e^{-x})$$

Saida entre 0 e 1 = probabilidade. Usada quando ha 2 classes (spam/nao, gato/cachorro).

Softmax

Saida Multiclasse

$$e^{x_i} / \sum e^{x_j}$$

Distribui probabilidade entre todas as classes (soma = 100%). Iris (3 classes), digitos (10 classes).

[Ferramenta: Activation Functions](#)

Arquitetura da Rede e Regularizacão

Arquitetura da Rede (Topologia)

O número de camadas ocultas e neurônios por camada.

Rede rasa (1-2 camadas):

Boa para problemas simples. Treino rápido, menos risco de overfitting.

Rede profunda (muitas camadas):

Captura padrões complexos. Mais poder, mas mais risco de overfitting e treino mais lento.

Dica prática:

Comece simples (1-2 hidden layers com 32-128 neurônios) e aumente se necessário.

Regularização

Técnicas para evitar overfitting, penalizando modelos muito complexos.

Dropout:

Desliga neurônios aleatórios durante o treino (ex: 20-50%). Força a rede a não depender de poucos neurônios.

L1 (Lasso):

Penaliza pesos pelo valor absoluto. Tende a zerar pesos menos importantes (seleção de features).

L2 (Ridge):

Penaliza pesos pelo quadrado. Mantém todos os pesos pequenos. O mais comum.

```
model.add(Dense(64, activation='relu')) → model.add(Dropout(0.3)) → model.add(Dense(3, activation='softmax'))
```